

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI THỰC HÀNH
HỌC PHẦN: PHÂN TÍCH MÃ ĐỘC
MÃ HỌC PHẦN: INT14164**

BÀI THỰC HÀNH 4

**Phân tích tĩnh mã độc Trojan Downloader Multiverze nâng cao trên
Linux**

Sinh viên thực hiện:

B22DCAT009 Đàm Quốc Anh

Giảng viên hướng dẫn: TS.Đình Trường Dục

HỌC KỲ 1 NĂM HỌC 2025-2026

MỤC LỤC

MỤC LỤC.....	2
DANH MỤC CÁC HÌNH VẼ.....	3
CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH	4
1.1 Mục đích.....	4
CHƯƠNG 2. NỘI DUNG THỰC HÀNH	4
2.1 Các bước thực hiện.....	4
2.1.1 Nhiệm vụ 1: Kiểm tra các byte và biểu diễn theo ký tự ASCII	4
2.1.2 Nhiệm vụ 2: Kiểm tra các Section	5
2.1.3 Nhiệm vụ 3 :Kiểm tra Architecture	6
2.1.4 Nhiệm vụ 4 :Hành vi kiểm tra sự tồn tại của tệp lạ nào?	7
2.1.5 Nhiệm vụ 5: Xác định tên miền hoặc địa chỉ IP mà mã độc thực hiện kết nối.?	7
2.1.6 Nhiệm vụ 6: Xác định hành vi khi cố gắng kết nối đến một máy chủ từ xa?	8
2.1.7 Nhiệm vụ 7: Gửi dữ liệu gì(chuỗi) qua mạng là chuỗi nào?	8
2.2 Checkwork	8

DANH MỤC CÁC HÌNH VẼ

Hình 1 Kiểm tra các byte.....	4
Hình 2 Danh sách các section	5
Hình 3 Xem nội dung của section .rodata	5
Hình 4 Kiến trúc của tệp madoc.elf.....	6
Hình 5 Xem file nhị phân.....	6
Hình 6 Xem tất cả các hàm	7
Hình 7 Chuỗi liên quan tới file.....	7
Hình 8 Chuỗi liên quan tới tên miền hoặc địa chỉ IP	7
Hình 9 Chuỗi chứng minh thực hiện kết nối mạng	8
Hình 10 Chuỗi liên quan tới truyền dữ liệu.....	8
Hình 11 Checkwork	8

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ BÀI THỰC HÀNH

1.1 Mục đích

Giúp sinh viên nắm vững cách thực hiện phân tích tĩnh mã độc trên Linux, đặc biệt là mã độc loại Trojan Downloader. Đồng thời, rèn luyện kỹ năng sử dụng các công cụ như hexdump, objdump, và radare2 trong quy trình phân tích tĩnh, nhằm xác định cấu trúc, chức năng, và hành vi tiềm năng của mã độc mà không cần thực thi.

CHƯƠNG 2. NỘI DUNG THỰC HÀNH

2.1 Các bước thực hiện

Khởi động bài lab:

Sinh viên tải bài thực hành bằng lệnh:

```
imodule https://github.com/duydt2024/-ATNN_D20/raw/main/mal-stic-lin-troj-dl-mtz-r2/imodule.tar
```

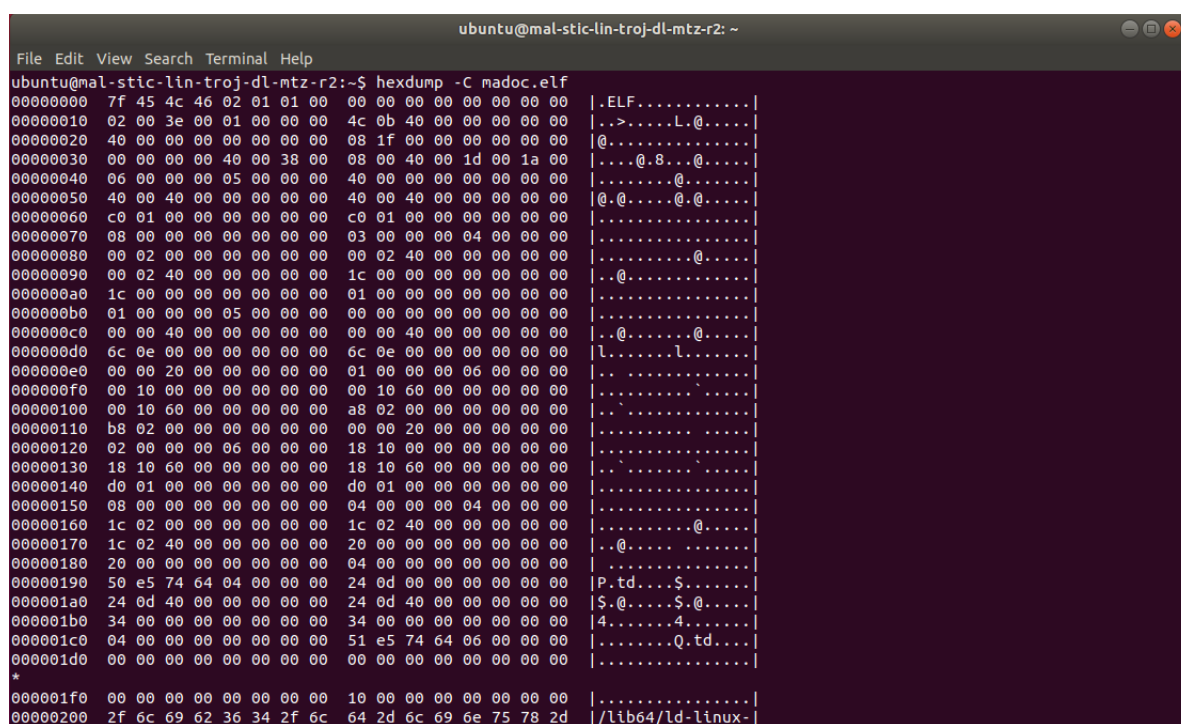
Bắt đầu bài lab ,vào Terminal, gõ: `labtainer -r mal-stic-lin-troj-dl-mtz-r2`

Sau khi khởi động xong một terminal ảo sẽ xuất hiện, đại diện cho máy `mal-stic-lin-troj-dl-mtz-r2`, bên trong bài lab đã có sẵn mã độc “`madoc.elf`”

2.1.1 Nhiệm vụ 1: Kiểm tra các byte và biểu diễn theo ký tự ASCII

Sử dụng lệnh ”`hexdump`” kiểm tra các byte

“-C” biểu diễn theo ký tự ASCII



```
ubuntu@mal-stic-lin-troj-dl-mtz-r2: ~
File Edit View Search Terminal Help
ubuntu@mal-stic-lin-troj-dl-mtz-r2:~$ hexdump -C madoc.elf
00000000  7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00  |.ELF.....|
00000010  02 00 3e 00 01 00 00 00 4c 0b 40 00 00 00 00 00  |..>....L.@...|
00000020  40 00 00 00 00 00 00 00 08 1f 00 00 00 00 00 00  |@.....|
00000030  00 00 00 00 40 00 38 00 08 00 40 00 1d 00 1a 00  |....@.8.....|
00000040  06 00 00 00 05 00 00 00 40 00 00 00 00 00 00 00  |.....@.....|
00000050  40 00 40 00 00 00 00 00 40 00 40 00 00 00 00 00  |@.@.....@.@...|
00000060  c0 01 00 00 00 00 00 00 c0 01 00 00 00 00 00 00  |.....|
00000070  08 00 00 00 00 00 00 00 03 00 00 00 04 00 00 00  |.....|
00000080  00 02 00 00 00 00 00 00 00 02 40 00 00 00 00 00  |.....@.....|
00000090  00 02 40 00 00 00 00 00 1c 00 00 00 00 00 00 00  |..@.....|
000000a0  1c 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00  |.....|
000000b0  01 00 00 00 05 00 00 00 00 00 00 00 00 00 00 00  |.....|
000000c0  00 00 40 00 00 00 00 00 00 00 40 00 00 00 00 00  |..@.....@.....|
000000d0  6c 0e 00 00 00 00 00 00 6c 0e 00 00 00 00 00 00  |l.....l.....|
000000e0  00 00 20 00 00 00 00 00 01 00 00 00 06 00 00 00  |.....|
000000f0  00 10 00 00 00 00 00 00 00 10 60 00 00 00 00 00  |.....|
00000100  00 10 60 00 00 00 00 00 a8 02 00 00 00 00 00 00  |.....|
00000110  b8 02 00 00 00 00 00 00 00 00 20 00 00 00 00 00  |.....|
00000120  02 00 00 00 06 00 00 00 18 10 00 00 00 00 00 00  |.....|
00000130  18 10 60 00 00 00 00 00 18 10 60 00 00 00 00 00  |.....|
00000140  d0 01 00 00 00 00 00 00 d0 01 00 00 00 00 00 00  |.....|
00000150  08 00 00 00 00 00 00 00 04 00 00 00 04 00 00 00  |.....|
00000160  1c 02 00 00 00 00 00 00 1c 02 40 00 00 00 00 00  |.....@.....|
00000170  1c 02 40 00 00 00 00 00 20 00 00 00 00 00 00 00  |..@.....|
00000180  20 00 00 00 00 00 00 00 04 00 00 00 00 00 00 00  |.....|
00000190  50 e5 74 64 04 00 00 00 24 0d 00 00 00 00 00 00  |P.td....$.|
000001a0  24 0d 40 00 00 00 00 00 24 0d 40 00 00 00 00 00  |$.@....$.@....|
000001b0  34 00 00 00 00 00 00 00 34 00 00 00 00 00 00 00  |4.....4.....|
000001c0  04 00 00 00 00 00 00 00 51 e5 74 64 06 00 00 00  |.....Q.td....|
000001d0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  |.....|
*
000001f0  00 00 00 00 00 00 00 00 10 00 00 00 00 00 00 00  |.....|
00000200  2f 6c 69 62 36 34 2f 6c 64 2d 6c 69 6e 75 78 2d  |/lib64/ld-linux-|
```

Hình 1 Kiểm tra các byte

2.1.2 Nhiệm vụ 2: Kiểm tra các Section

Sử dụng “objdump” với tùy chọn “-h” để hiển thị danh sách các section

Sinh viên cần đọc và kiểm tra tất cả các section

```
ubuntu@mal-stic-lin-troj-dl-mtz-r2: ~  
File Edit View Search Terminal Help  
ubuntu@mal-stic-lin-troj-dl-mtz-r2:~$ objdump -h madoc.elf  
madoc.elf:      file format elf64-x86-64  
  
Sections:  
Idx Name              Size      VMA              LMA              File off  Algn  
 0 .interp             0000001c 0000000000400200 0000000000400200 00000200 2**0  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 1 .note.ABI-tag       00000020 000000000040021c 000000000040021c 0000021c 2**2  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 2 .hash               000000a4 0000000000400240 0000000000400240 00000240 2**3  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 3 .dynsym             00000210 00000000004002e8 00000000004002e8 000002e8 2**3  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 4 .dynstr             000000c6 00000000004004f8 00000000004004f8 000004f8 2**0  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 5 .gnu.version        0000002c 00000000004005be 00000000004005be 000005be 2**1  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 6 .gnu.version_r     00000030 00000000004005f0 00000000004005f0 000005f0 2**3  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 7 .rela.dyn           00000030 0000000000400620 0000000000400620 00000620 2**3  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 8 .rela.plt           000001b0 0000000000400650 0000000000400650 00000650 2**3  
   CONTENTS, ALLOC, LOAD, READONLY, DATA  
 9 .init               0000001a 0000000000400800 0000000000400800 00000800 2**2  
   CONTENTS, ALLOC, LOAD, READONLY, CODE  
10 .plt                00000130 0000000000400820 0000000000400820 00000820 2**4  
   CONTENTS, ALLOC, LOAD, READONLY, CODE  
11 .text               00000372 0000000000400950 0000000000400950 00000950 2**4  
   CONTENTS, ALLOC, LOAD, READONLY, CODE  
12 .fini               00000009 0000000000400cc4 0000000000400cc4 00000cc4 2**2  
   CONTENTS, ALLOC, LOAD, READONLY, CODE  
13 .rodata             00000051 0000000000400cd0 0000000000400cd0 00000cd0 2**4  
   CONTENTS, ALLOC, LOAD, READONLY, DATA
```

Hình 2 Danh sách các section

Sử dụng “objdump” với các tùy chọn:

- -s: Hiển thị nội dung nhị phân của section.
- -j <tên section>: Chỉ định section cụ thể cần kiểm tra.

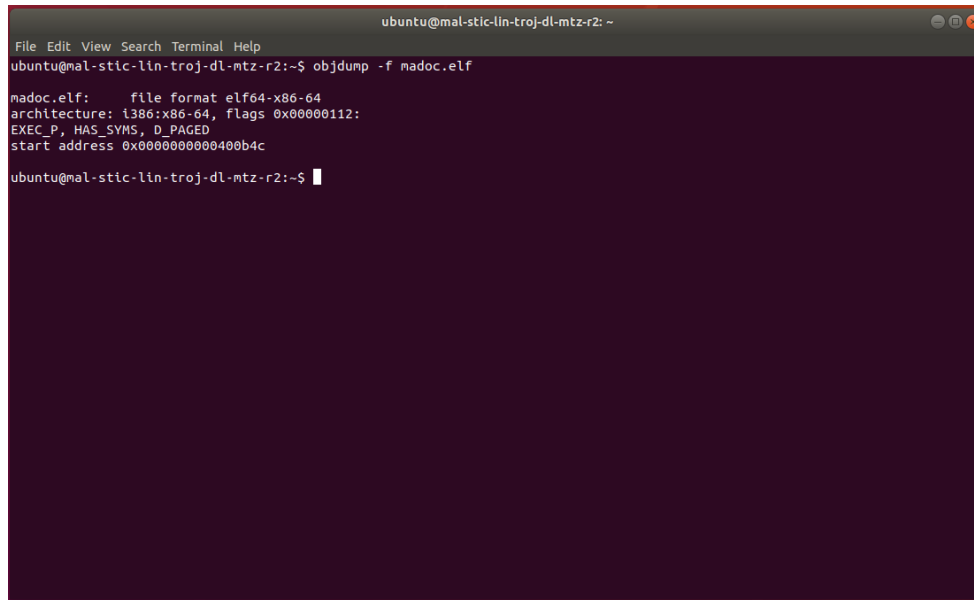
```
ubuntu@mal-stic-lin-troj-dl-mtz-r2: ~  
File Edit View Search Terminal Help  
ubuntu@mal-stic-lin-troj-dl-mtz-r2:~$ objdump -s -j .rodata madoc.elf  
madoc.elf:      file format elf64-x86-64  
  
Contents of section .rodata:  
400cd0 01000200 2f746d70 2f6c6f67 5f64652e  ....tmp/log_de.  
400ce0 6c6f6700 6c363420 20200061 00435744  log.164      .a.CWD  
400cf0 005b6b77 6f726b65 722f303a 325d0000  .[kworker/0:2]..  
400d00 7777772e 67616f67 616f7a69 2e636f6d  www.gaogaozi.com  
400d10 00000000 00000000 00000000 00000000  ..  
400d20 00  
ubuntu@mal-stic-lin-troj-dl-mtz-r2:~$
```

Hình 3 Xem nội dung của section .rodata

2.1.3 Nhiệm vụ 3 :Kiểm tra Architecture

Sử dụng “objdump” với tùy chọn “-f” để kiểm tra kiến trúc của tệp madoc.elf

- Tập này có kiến trúc gì? (x86, x86_64, ARM, v.v.)
- Kiến trúc này có ảnh hưởng gì đến cách mã thực thi và ảnh hưởng đến các hệ thống bị nhiễm mã độc?



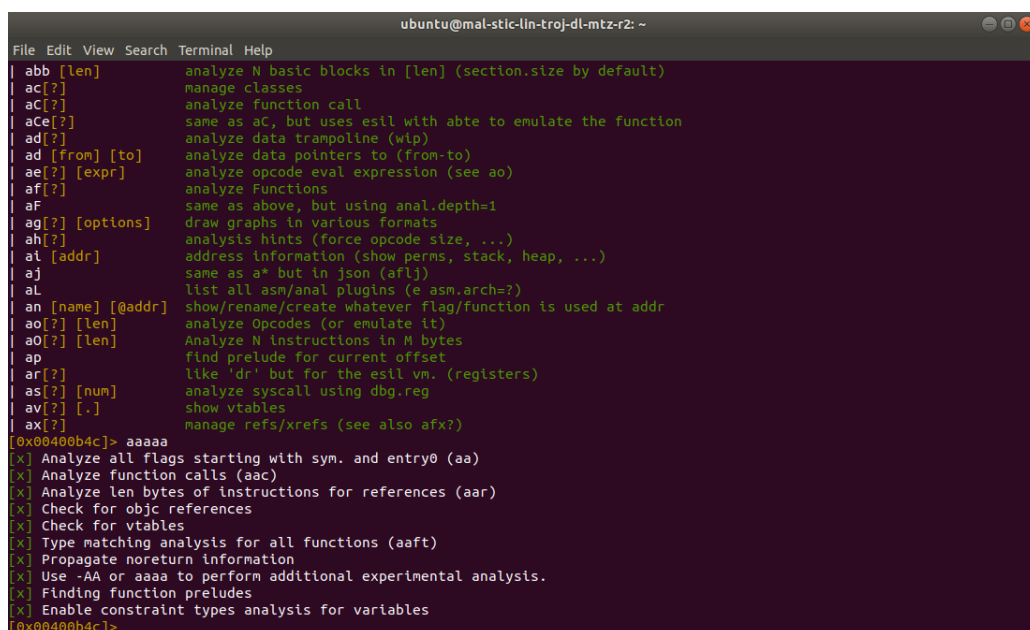
```
ubuntu@mal-stic-lin-troj-dl-mtz-r2: ~  
File Edit View Search Terminal Help  
ubuntu@mal-stic-lin-troj-dl-mtz-r2:~$ objdump -f madoc.elf  
  
madoc.elf:      file format elf64-x86-64  
architecture: i386:x86-64, flags 0x00000112:  
EXEC_P, HAS_SYMS, D_PAGED  
start address 0x00000000400b4c  
  
ubuntu@mal-stic-lin-troj-dl-mtz-r2:~$
```

Hình 4 Kiến trúc của tệp madoc.elf

Sử dụng công cụ Radare2 để tìm chức năng chính của mẫu dùng lệnh:

r2 madoc.elf

Lệnh “aaaaa”: Tự động phân tích một file nhị phân hoặc một đoạn mã.



```
ubuntu@mal-stic-lin-troj-dl-mtz-r2: ~  
File Edit View Search Terminal Help  
| abb [len]          analyze N basic blocks in [len] (section.size by default)  
| ac[?]             manage classes  
| aC[?]            analyze function call  
| aCe[?]           same as ac, but uses esil with abte to emulate the function  
| ad[?]           analyze data trampoline (wip)  
| ad [from] [to]    analyze data pointers to (from-to)  
| ae[?] [expr]      analyze opcode eval expression (see ao)  
| af[?]           analyze Functions  
| aF             same as above, but using anal.depth=1  
| ag[?] [options]  draw graphs in various formats  
| ah[?]           analysis hints (force opcode size, ...)  
| al [addr]        address information (show perms, stack, heap, ...)  
| aj             same as a* but in json (ajl)  
| al            list all asm/anal plugins (e asm.arch=?)  
| an [name] [addr] show/rename/create whatever flag/function is used at addr  
| ao[?] [len]      analyze Opcodes (or emulate it)  
| aO[?] [len]      Analyze N instructions in M bytes  
| ap            find prelude for current offset  
| ar[?]         like 'dr' but for the esil vm. (registers)  
| as[?] [num]     analyze syscall using dbg.reg  
| av[?] [...]     show vtables  
| ax[?]          manage refs/xrefs (see also afx?)  
[0x00400b4c]> aaaaa  
[x] Analyze all flags starting with sym. and entry0 (aa)  
[x] Analyze function calls (aac)  
[x] Analyze len bytes of instructions for references (aar)  
[x] Check for objc references  
[x] Check for vtables  
[x] Type matching analysis for all functions (aaft)  
[x] Propagate noreturn information  
[x] Use -AA or aaaa to perform additional experimental analysis.  
[x] Finding function preludes  
[x] Enable constraint types analysis for variables  
[0x00400b4c]>
```

Hình 5 Xem file nhị phân

Liệt kê tất cả các hàm (Analysis Functions List), sử dụng các lệnh: “afl”: để xem thông tin của tất cả các hàm, “s <địa chỉ của hàm>”: để di chuyển tới vùng nhớ hiện tại đến địa chỉ bắt đầu của hàm, “pdf”: để hiển thị mã máy của hàm. Sinh viên cần đọc các hàm để hoàn thành các câu hỏi hành vi ở nhiệm vụ 4,5,6,7 rồi in ra màn hình

```

ubuntu@mal-stic-lin-troj-dl-mtz-r2: ~
File Edit View Search Terminal Help
ubuntu@mal-stic-lin-troj-dl-mtz-r2:~$ r2 madoc.elf
[0x00400b4c]> afl
Usage: a [abdefFghoprxtsc] [...]
| a                alias for afl - analysis information
| a*              same as afl;ah*;ax*
| aa[?]          analyze all (fcns + bbs) (aa0 to avoid sub renaming)
| a8 [hexpairs]   analyze bytes
| ab[b] [addr]    analyze block at given address
| abb [len]       analyze N basic blocks in [len] (section.size by default)
| ac[?]          manage classes
| ac[?]          analyze function call
| aCe[?]         same as ac, but uses esil with abte to emulate the function
| ad[?]          analyze data trampoline (wlp)
| ad [from] [to]  analyze data pointers to (from-to)
| ae[?] [expr]    analyze opcode eval expression (see ao)
| af[?]          analyze Functions
| af            same as above, but using anal.depth=1
| ag[?] [options] draw graphs in various formats
| ah[?]          analysis hints (force opcode size, ...)
| ai [addr]       address information (show perms, stack, heap, ...)
| aj            same as a* but in json (aflj)
| al            list all asm/anal plugins (e asm.arch=?)
| an [name] [@addr] show/rename/create whatever flag/function is used at addr
| ao[?] [len]     analyze Opcodes (or emulate it)
| aO[?] [len]     Analyze N instructions in M bytes
| ap            find prelude for current offset
| ar[?]          like 'dr' but for the esil VM. (registers)
| as[?] [num]     analyze syscall using dbg.reg
| av[?] [.]       show vtables
| ax[?]          manage refs/xrefs (see also afx?)
[0x00400b4c]> aaaaa
[x] Analyze all flags starting with sym. and entry0 (aa)
[x] Analyze function calls (aac)
[x] Analyze len bytes of instructions for references (aar)

```

Hình 6 Xem tất cả các hàm

2.1.4 Nhiệm vụ 4 :Hành vi kiểm tra sự tồn tại của tệp lạ nào?

Tìm kiếm các hàm liên quan đến việc kiểm tra file.

Trong mã máy của hàm, sinh viên cần tìm các chuỗi liên quan đến file (thường được gọi bằng hàm fopen, access, hoặc stat).

```

0x0040095b    bfd40c4000    mov edi, str.tmp_log_de.log ; 0x400cd4 ; "/tmp/log_de.log" ; const char *path
0x00400960    4881ec581400. sub rsp, 0x1458
0x00400967    e894ffff     call sym.imp.access ; int access(const char *path, int mode)

```

Hình 7 Chuỗi liên quan tới file

2.1.5 Nhiệm vụ 5: Xác định tên miền hoặc địa chỉ IP mà mã độc thực hiện kết nối.?

Tìm và phân tích các hàm thực hiện kết nối mạng, như gethostbyname, inet_addr, hoặc connect.

Tìm tên miền hoặc địa chỉ IP được sử dụng

```

0x0040097c    be000d4000    mov esi, str.www.gaogaozi.com ; 0x400d00 ; "www.gaogaozi.com"
0x00400981    b921000000    mov ecx, 0x21 ; '!' ; 33
0x00400986    f3a4          rep movsb byte [rdi], byte ptr [rsi]
0x00400988    488d7c242f    lea rdi, qword [var_1449h]
0x0040098d    48c744241c00. mov qword [addr], 0
0x00400996    48c744242400. mov qword [var_1454h], 0
0x0040099f    66c744241c02. mov word [addr], 2
0x004009a6    66c744241e1f. mov word [var_145ah], 0x471f ; [0x471f:2]=0xffff
0x004009ad    e8fefeffff    call sym.imp.gethostbyname

```

Hình 8 Chuỗi liên quan tới tên miền hoặc địa chỉ IP

2.1.6 Nhiệm vụ 6: Xác định hành vi khi cố gắng kết nối đến một máy chủ từ xa?

Sinh viên cần nhận ra rằng việc gọi hàm nào là bằng chứng của việc mã độc thực hiện kết nối mạng với một máy chủ từ xa.

```
0x00400a1b e800ffffff call sym.imp.connect ; ssize_t connect(int socket, void *addr, size_t addrlen)
0x00400a20 ffca
```

Hình 9 Chuỗi chứng minh thực hiện kết nối mạng

2.1.7 Nhiệm vụ 7: Gửi dữ liệu gì(chuỗi) qua mạng là chuỗi nào?

Sinh viên cần xác định dữ liệu mà mã độc gửi qua mạng. Điều này thường được thực hiện thông qua việc gọi hàm send hoặc các hàm tương tự để truyền dữ liệu (thường là chuỗi) đến máy chủ từ xa

```
0x00400a37 bee40c4000 mov esi, str.l64 ; 0x400ce4 ; "l64 " ; void *buffer
0x00400a3c 89df mov edi, ebx ; int socket
0x00400a3e e82dfeffff call sym.imp.send ; ssize_t send(int socket, void *buffer, size_t length, int flags)
```

Hình 10 Chuỗi liên quan tới truyền dữ liệu

Sau đó, sinh viên sử dụng lệnh cat với file FileToView.txt để hoàn thành bài lab.

2.2 Checkwork

```
student@ubuntu:~/labtainer/labtainer-student$ checkwork
Results stored in directory: /home/student/labtainer_xfer/mal-stic-lln-troj-dl-mtz-r2
Labname mal-stic-lln-troj-dl-mtz-r2

Student | Hexdump | Section | Architecture | Analy_Function | File_log | Domain | Connect_network | Send |
=====|=====|=====|=====|=====|=====|=====|=====|=====|
B22DCAT009 | Y | Y | Y | Y | Y | Y | Y | Y |
What is automatically assessed for this lab:

student@ubuntu:~/labtainer/labtainer-student$
```

Hình 11 Checkwork